ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA KỸ THUẬT MÁY TÍNH



BÁO CÁO BÀI TẬP

MÔN HỌC: THIẾT KẾ LUẬN LÝ SỐ

ĐỀ TÀI

THIẾT KẾ MẠCH TUẦN TỰ NHÂN HAI SỐ NHỊ PHÂN 4 BIT

Giảng viên hướng dẫn: Hồ Ngọc Diễm
Sinh viên thực hiện: Huỳnh Phạm Ngọc Thắng
Mã số sinh viên: 25521672
Lớp: CE118.R11.VMTN

TP. Hồ Chí Minh, tháng 10 năm 2026

1. GIỚI THIỆU ĐỀ TÀI

1.1. Yêu cầu bài tập

Thực hiện mạch tuần tự nhân hai số nhị phân 4 bit trên phần mềm Quartus, với các yêu cầu:

- Trình bày **lưu đồ ASM**, **Datapath** và **Control Unit** của mạch.
- **Vẽ mạch** trên Quartus (Block Diagram/Schematic), không dùng Verilog và không dùng IC/megafunction có sẵn.
- **Mô phỏng Functional và Timing**, kiểm tra kết quả và **số chu kỳ hoàn thành**.

1.2. Mô tả bài toán

Mạch nhận hai số nhị phân không dấu 4 bit: số bị nhân **InA[3..0]** và số nhân **InB[3..0]**, tính tích và xuất kết quả 8 bit ra **Tich[7..0]**. Khi tích đã sẵn sàng, mạch bật tín hiệu **Done = 1** trong một chu kỳ. Mạch được thiết kế theo mô hình **FSMD** (Finite State Machine with Datapath) gồm hai phần:

- **Datapath** (custom datapath): thanh ghi A 8 bit (dịch trái), thanh ghi B 4 bit (dịch phải), thanh ghi Tích 8 bit, bộ cộng 8 bit và bộ đếm vòng lặp cnt 2 bit.
- **Control Unit:** máy trạng thái 4 trạng thái (S0–S3) sinh các tín hiệu điều khiển Init, Run, LdT, Done cho datapath.

Bảng 1.1: Thông số thiết kế

Thông số	Giá trị
Độ rộng dữ liệu vào	4 bit (InA, InB)
Độ rộng kết quả	8 bit (Tich), giá trị lớn nhất 15 × 15 = 225
Thanh ghi datapath	A (8 bit), B (4 bit), Tich (8 bit), cnt (2 bit)
Số trạng thái FSM	4 (S0–S3), mã hóa nhị phân 2 bit Q_1Q_0
Tín hiệu vào	CLK, Start, InA[3..0], InB[3..0]
Tín hiệu ra	Tich[7..0], Done (báo kết quả hợp lệ)
Công cụ	Quartus II 13.0 SP1 (Block Diagram + Simulation Waveform Editor), Cyclone II

2. THUẬT TOÁN VÀ LƯU ĐỒ ASM

2.1. Thuật toán nhân cộng – dịch (shift-and-add)

Thuật toán mô phỏng cách nhân tay trên giấy: xét lần lượt từng bit của số nhân B, bắt đầu từ bit thấp nhất. Nếu bit đang xét bằng 1 thì cộng số bị nhân A vào tích, nếu bằng 0 thì giữ nguyên. Sau mỗi bước, A được dịch trái 1 bit (nhân 2, tương ứng với việc lùi một cột khi nhân tay) và B được dịch phải 1 bit để đưa bit kế tiếp xuống vị trí thấp nhất. Vì B có 4 bit nên vòng lặp chạy đúng **4 lần**.

Bảng 2.1: Thuật toán nhân hai số 4 bit

```
1. A    := {0000, InA[3..0]}     // mở rộng số bị nhân lên 8 bit
2. B    := InB[3..0]
3. Tich := 0000 0000
4. cnt  := 0
   repeat 4 lần
5.    if B[0] = 1 then Tich := Tich + A
6.    A := A << 1;  B := B >> 1;  cnt := cnt + 1
   end repeat
7. Outport := Tich;  Done := 1
```

Ví dụ với A = 1101 (13), B = 1010 (10), kết quả đúng là 13 × 10 = 130 = 1000 0010:

Bảng 2.2: Diễn biến thuật toán với 1101 × 1010

Vòng	B (đầu vòng)	B[0]	A (đầu vòng)	Thao tác	Tich sau vòng
Khởi tạo	1010	–	0000 1101	–	0000 0000 (0)
1	1010	0	0000 1101 (13)	Không cộng	0000 0000 (0)
2	0101	1	0001 1010 (26)	Tich + A	0001 1010 (26)
3	0010	0	0011 0100 (52)	Không cộng	0001 1010 (26)
4	0001	1	0110 1000 (104)	Tich + A	1000 0010 (130)

Các giá trị A = 13, 26, 52, 104 chính là các hàng "lùi cột" của phép nhân tay. Hàng nào ứng với bit B bằng 1 thì được cộng vào tích. Tích có 8 bit nên phép cộng không bao giờ tràn (lớn nhất 225 < 256), vì vậy không cần thanh ghi lưu bit nhớ.

2.2. Gán biến vào thanh ghi

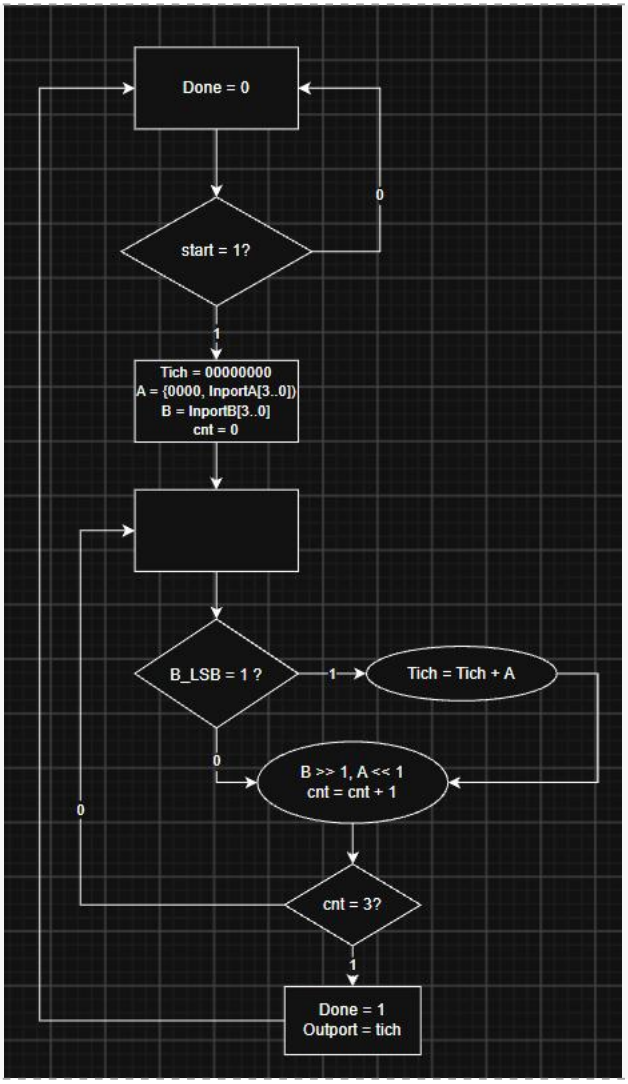
Mạch dùng **custom datapath**: mỗi biến trong thuật toán là một khối phần cứng riêng, chỉ có đúng những chức năng mà thuật toán cần.

Bảng 2.3: Bảng gán thanh ghi

Thanh ghi	Độ rộng	Biến	Chức năng cần có
RegA	8 bit	A (số bị nhân)	Load {0000, InA}, dịch trái, giữ nguyên
RegB	4 bit	B (số nhân)	Load InB, dịch phải, giữ nguyên
RegT	8 bit	Tich (tích)	Xóa về 0, nạp tổng Tich + A, giữ nguyên
cnt_eq_3	2 bit	cnt (bộ đếm vòng)	Xóa về 0, đếm lên, giữ nguyên; báo cnt = 3

2.3. Lưu đồ ASM

Lưu đồ ASM gồm 4 trạng thái. Vòng lặp được gộp vào **một trạng thái S2** theo kiểu **Mealy**: phép cộng nằm trong hộp điều kiện (hình oval) chỉ thực hiện khi B_LSB = 1, còn phép dịch A, B và tăng cnt thực hiện ở mọi chu kỳ của S2. Nhờ vậy mỗi vòng lặp chỉ tốn đúng 1 chu kỳ clock.



Hình 2.1: Lưu đồ ASM của mạch nhân 4 bit

Bảng 2.4: Bảng trạng thái và hoạt động datapath

Trạng thái	Q_1Q_0	Điều kiện → Trạng thái kế	Điều kiện	Thao tác datapath
S0	00	Start = 0 → S0 Start = 1 → S1		Done = 0
S1	01	→ S2		Tich = 0000 0000 A = {0000, InA[3..0]} B = InB[3..0] cnt = 0
S2	10	cnt = 3 → S3 cnt ≠ 3 → S2	B_LSB = 1 –	Tich = Tich + A B >> 1, A << 1, cnt = cnt + 1
S3	11	→ S0		Done = 1 Outport = Tich

Lưu ý về điều kiện thoát cnt = 3: mọi phép gán trong một ASM block chỉ có hiệu lực tại cạnh clock cuối chu kỳ, nên hộp quyết định "cnt = 3?" đọc **giá trị cnt cũ** (trước khi tăng). Ở vòng thứ 4, cnt cũ bằng 3 nên mạch thoát sang S3, vòng lặp chạy đúng 4 lần. Nếu so sánh với 4, mạch sẽ lặp 5 lần và cnt phải có 3 bit. Tương tự, hộp "B_LSB = 1?" đọc B trước khi dịch, và phép cộng Tich + A dùng A trước khi dịch.

3. THIẾT KẾ DATAPATH

3.1. Cấu trúc tổng quát và tín hiệu điều khiển

Datapath gồm thanh ghi A, B, Tích, bộ cộng 8 bit và bộ đếm cnt. Ngõ ra của A và Tích đưa vào bộ cộng, tổng **Sum[7..0]** quay về ngõ vào thanh ghi Tích. Bit thấp nhất của B (**B0**) và cờ **cnt_eq_3** được gửi về Control Unit.

Theo bảng trạng thái, các phép gán xảy ra cùng một trạng thái có thể dùng chung một tín hiệu điều khiển. Vì vậy 9 tín hiệu ban đầu được gộp lại còn 4:

Bảng 3.1: Gộp tín hiệu điều khiển

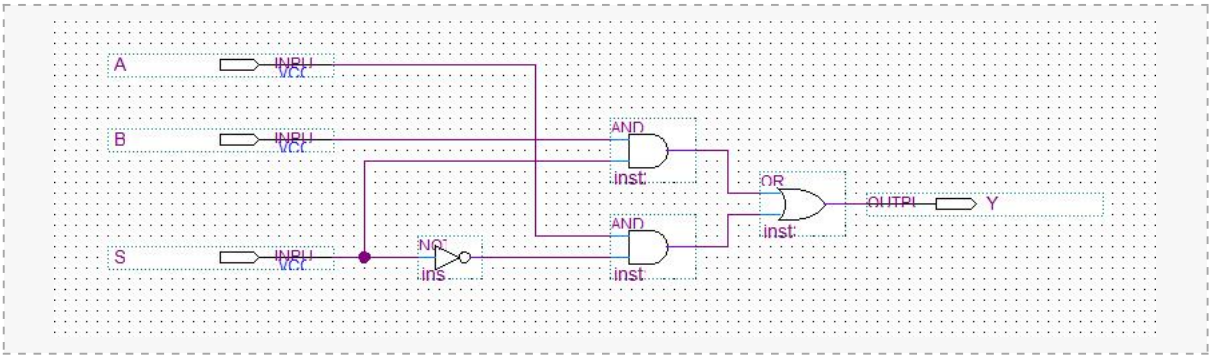
Tín hiệu	Thay cho	Phép gán tương ứng	Bật khi
Init	LdA, LdB, ClrT, ClrC	A = {0000, InA}, B = InB, Tich = 0, cnt = 0	S1
Run	ShA, ShB, EnC	A << 1, B >> 1, cnt = cnt + 1	S2
LdT	LdT	Tich = Tich + A	S2 và B0 = 1
Done	Done	Done = 1	S3

3.2. Các khối cơ bản

3.2.1. MUX 2-1 (mux2_1)

$$Y = S' \cdot A + S \cdot B$$

S = 0 → Y = A; S = 1 → Y = B. Mạch gồm 1 cổng NOT, 2 cổng AND và 1 cổng OR. Khối này được dùng trong các thanh ghi A, B và Tích để chọn giá trị nạp vào D flip-flop.

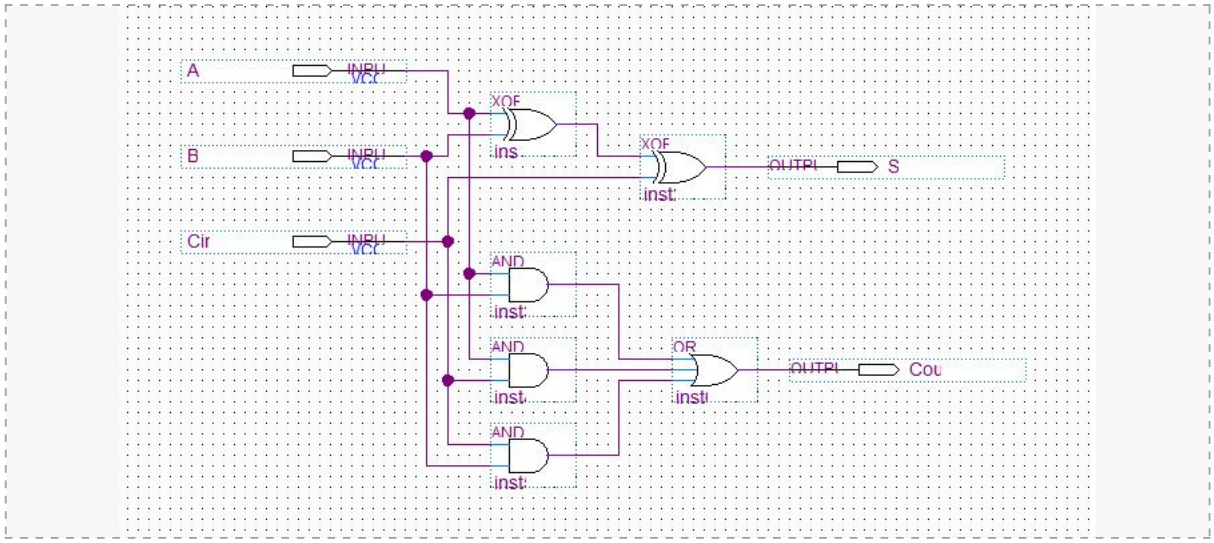


Hình 3.1: Mạch MUX 2-1

3.2.2. Full adder

$$S = A \oplus B \oplus C_{in}$$

$$C_{out} = A \cdot B + A \cdot C_{in} + B \cdot C_{in}$$

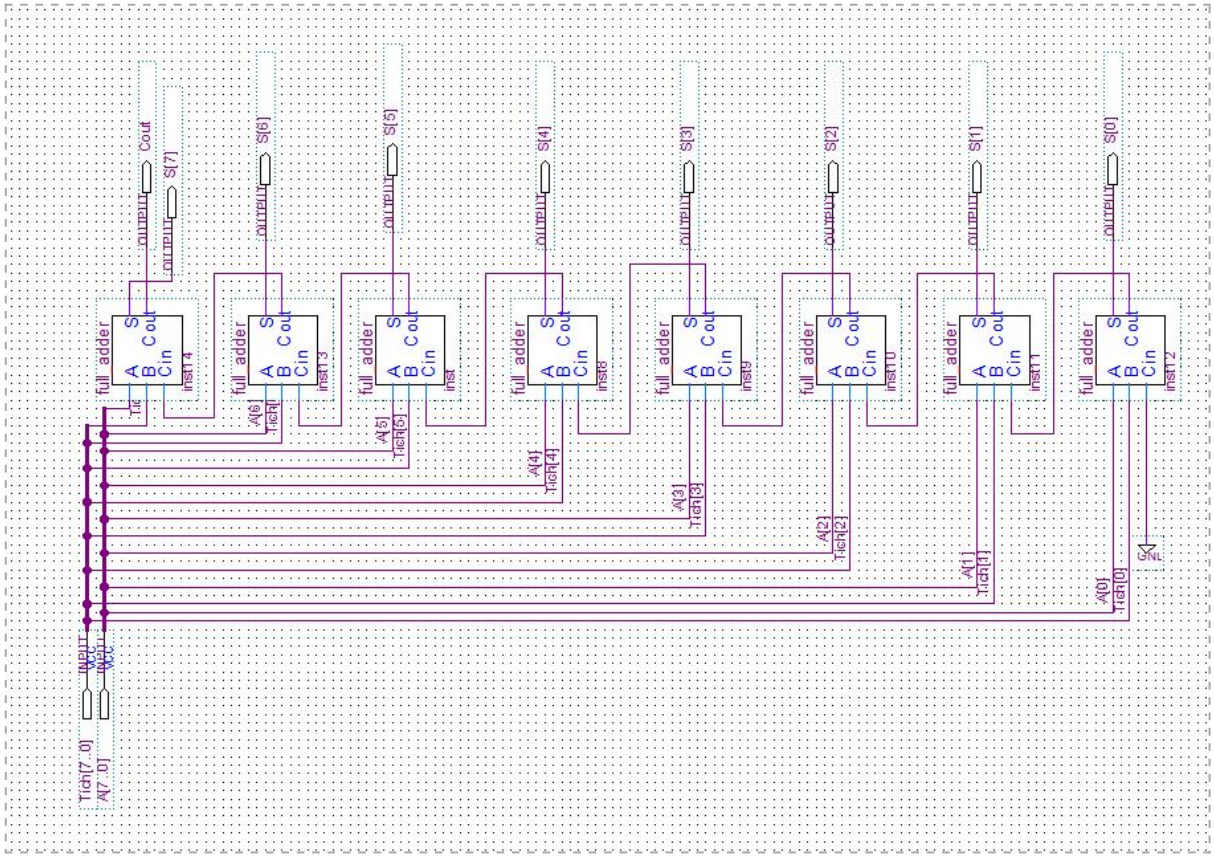


Hình 3.2: Mạch Full adder

3.2.3. Bộ cộng 8 bit (full_adder_8bit)

Ghép nối tiếp (ripple carry) 8 full adder: C_{out} của bit i nối vào C_{in} của bit i+1, C_{in} của bit 0 nối GND. Ngõ vào là **Tich[7..0]** và **A[7..0]**, ngõ ra **S[7..0]**. C_{out} của bit 7 không dùng vì tích lớn nhất (225) vẫn nằm trong 8 bit.

Sum[7..0] = Tich[7..0] + A[7..0]



Hình 3.3: Bộ cộng 8 bit (full_adder_8bit)

3.3. Thanh ghi A 8 bit (RegA)

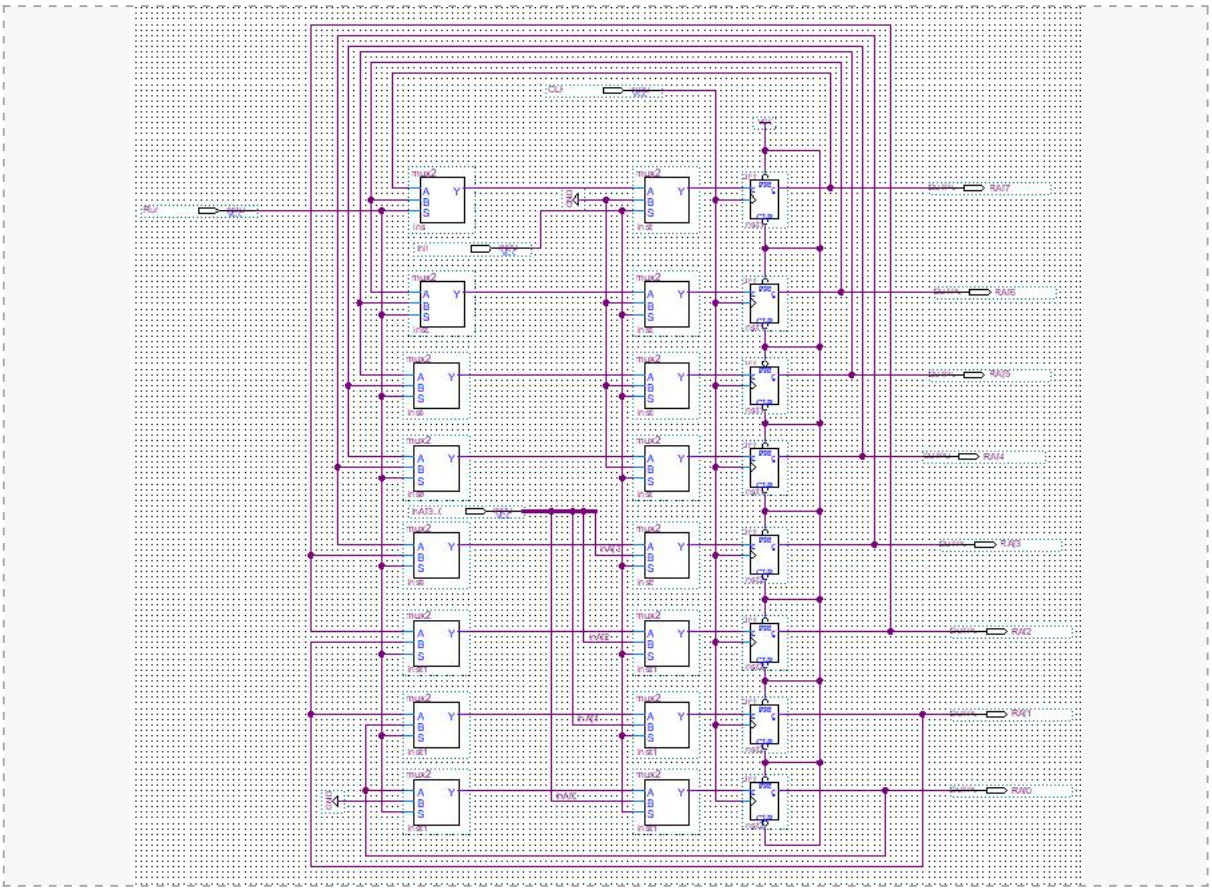
Thanh ghi A có 3 chức năng: load, dịch trái và giữ nguyên. Mỗi bit gồm **2 MUX 2-1 nối tiếp** và 1 D flip-flop: MUX thứ nhất do **Run** chọn giữa giữ nguyên (A_i) và dịch trái (A_{i-1}); MUX thứ hai do **Init** chọn giá trị load. Vì Init và Run không bao giờ cùng bằng 1 nên thứ tự ưu tiên không ảnh hưởng.

Bảng 3.2: Bảng chức năng thanh ghi A

Init	Run	D_i (bit 0)	D_i (bit 1–3)	D_i (bit 4–7)	Chức năng
0	0	A_0	A_i	A_i	Giữ nguyên
0	1	0 (GND)	A_{i-1}	A_{i-1}	Dịch trái (A << 1)
1	x	InA_0	InA_i	0 (GND)	Load {0000, InA}

$$A_i(next) = Init \cdot InA_i + Init' \cdot (Run' \cdot A_i + Run \cdot A_{i-1})$$

PRN và CLRN của các D flip-flop nối VCC. Ngõ ra RA[7..0] vừa hồi tiếp về các MUX, vừa đưa ra bộ cộng.



Hình 3.4: Thanh ghi A 8 bit (RegA)

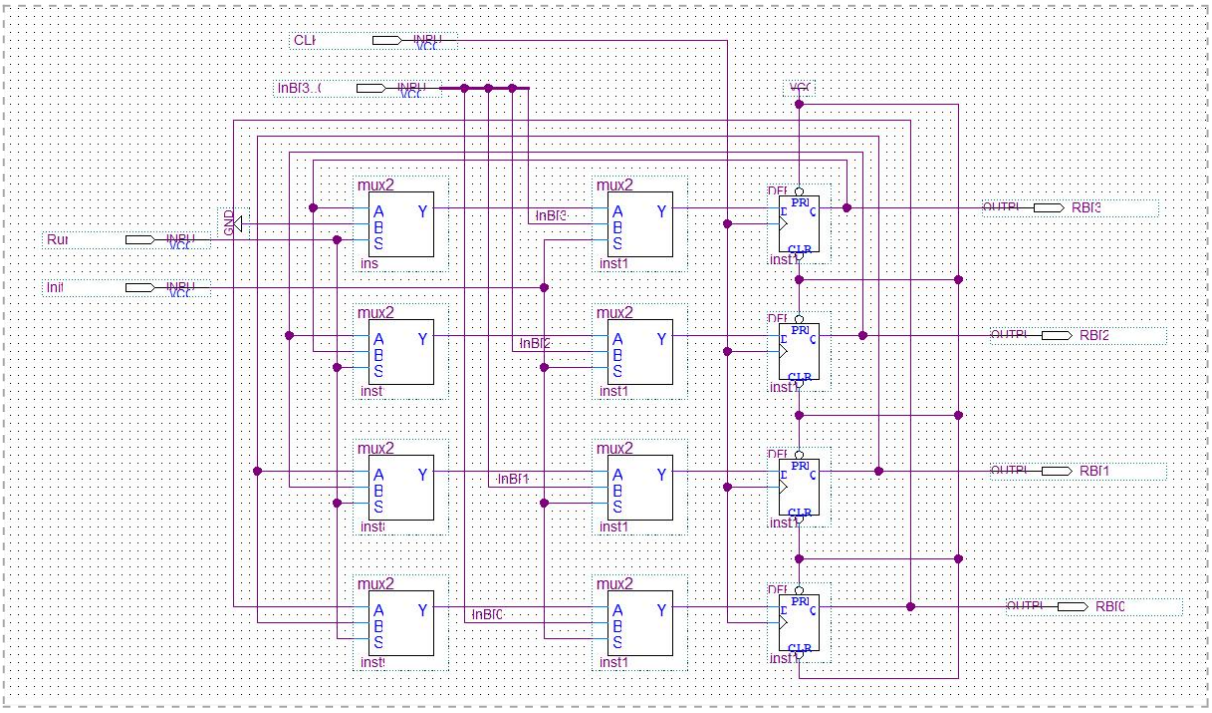
3.4. Thanh ghi B 4 bit (RegB)

Cấu trúc giống thanh ghi A nhưng dịch phải: khi Run = 1, bit i nhận giá trị bit i+1, bit 3 nhận 0 (GND). Ngõ ra RB[0] chính là tín hiệu **B0** gửi về Control Unit.

Bảng 3.3: Bảng chức năng thanh ghi B

Init	Run	D_3	D_i (bit 0–2)	Chức năng
0	0	B_3	B_i	Giữ nguyên
0	1	0 (GND)	B_{i+1}	Dịch phải (B >> 1)
1	x	InB_3	InB_i	Load InB

$$B_i(next) = Init \cdot InB_i + Init' \cdot (Run' \cdot B_i + Run \cdot B_{i+1}), \quad B_4 = 0$$



Hình 3.5: Thanh ghi B 4 bit (RegB)

3.5. Thanh ghi Tích 8 bit (RegT)

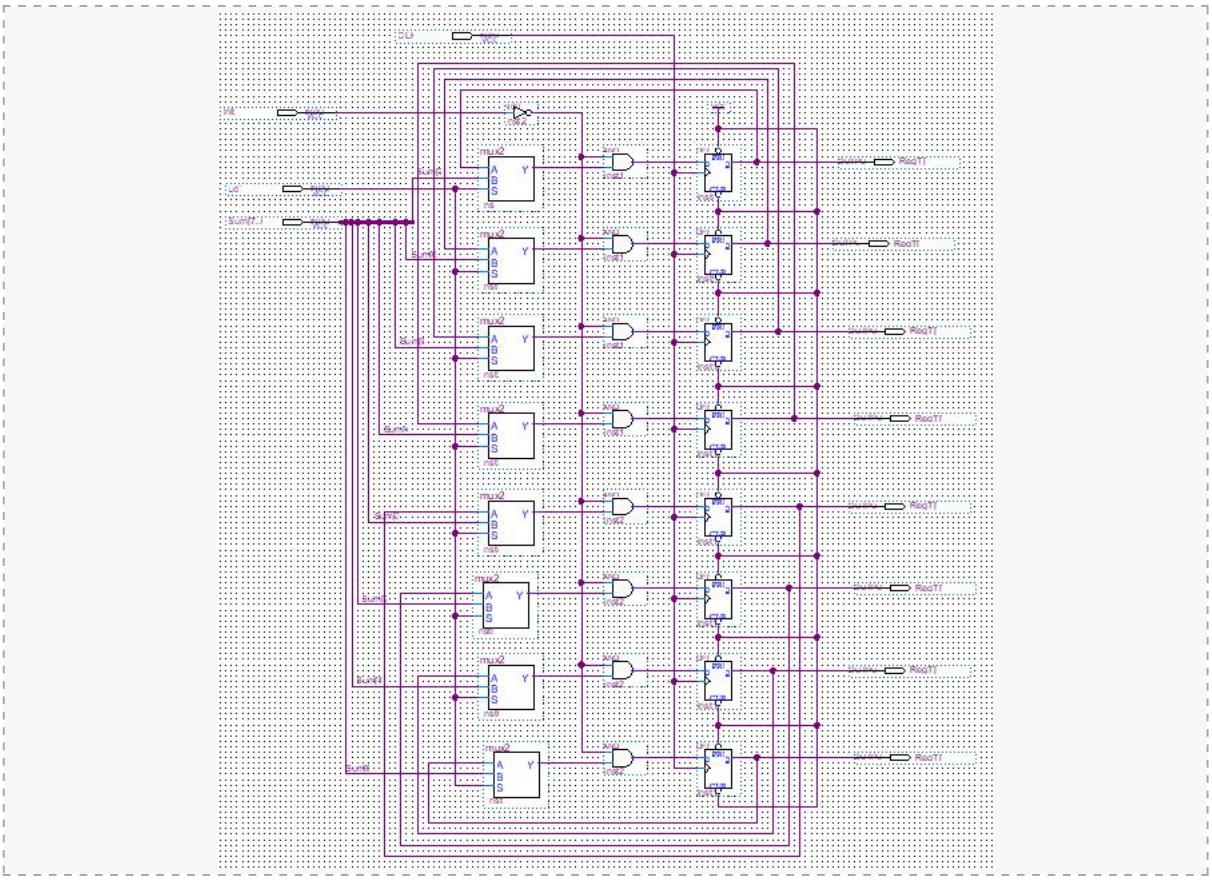
Thanh ghi Tích có 3 chức năng: xóa, nạp tổng và giữ nguyên. Mỗi bit gồm 1 MUX 2-1 (S = **LdT**, A = giữ nguyên T_i, B = Sum_i) và 1 cổng AND với **Init'** đặt trước chân D để xóa đồng bộ.

Bảng 3.4: Bảng chức năng thanh ghi Tích

Init	LdT	D_i	Chức năng
1	x	0	Xóa (Tich = 0)
0	1	Sum_i	Nạp tổng (Tich = Tich + A)
0	0	T_i	Giữ nguyên

$$T_i(next) = Init' \cdot (LdT' \cdot T_i + LdT \cdot Sum_i)$$

Vì sao xóa đồng bộ qua chân D mà không dùng CLRN: CLRN là xóa bất đồng bộ (tác động ngay, không chờ clock). Khi mô phỏng Timing, mạch giải mã Init = $Q_1'Q_0$ có thể sinh glitch lúc chuyển trạng thái (ví dụ S3 = 11 → S0 = 00 có thể thoáng qua 01) và xóa nhầm kết quả. Xóa qua chân D chỉ có hiệu lực tại cạnh clock nên an toàn.



Hình 3.6: Thanh ghi Tích 8 bit (RegT)

3.6. Bộ đếm vòng lặp cnt (cnt_eq_3)

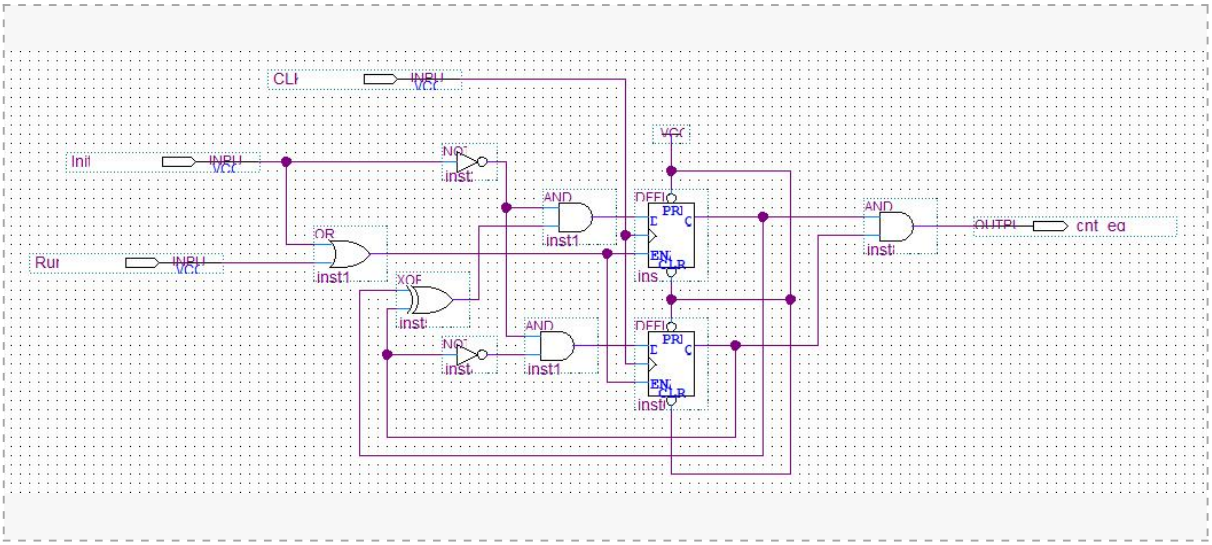
Bộ đếm 2 bit dùng 2 flip-flop **DFFE** (D flip-flop có chân cho phép ENA). Khi ENA = 0, DFFE giữ nguyên giá trị nên không cần mạch giữ riêng.

$$D_{c0} = Init' \cdot c_0' \qquad D_{c1} = Init' \cdot (c_1 \oplus c_0)$$
$$ENA = Init + Run \qquad cnt_eq_3 = c_1 \cdot c_0$$

Bảng 3.5: Bảng chức năng bộ đếm

Init	Run	ENA	D	Chức năng
1	0	1	00	Xóa về 0 (cnt = 0)
0	1	1	cnt + 1	Đếm lên
0	0	0	x	Giữ nguyên

Lưu ý: ENA phải là **Init + Run**, không chỉ là Run. DFFE chỉ nạp giá trị ở chân D khi ENA = 1; nếu ENA = Run thì ở S1 (Run = 0) giá trị 0 ở chân D bị bỏ qua và bộ đếm không được xóa. Mạch có thể vẫn chạy đúng do cnt tự tràn từ 3 về 0 sau vòng 4, nhưng khi đó phép gán cnt = 0 ở S1 không được thực hiện như ASM.

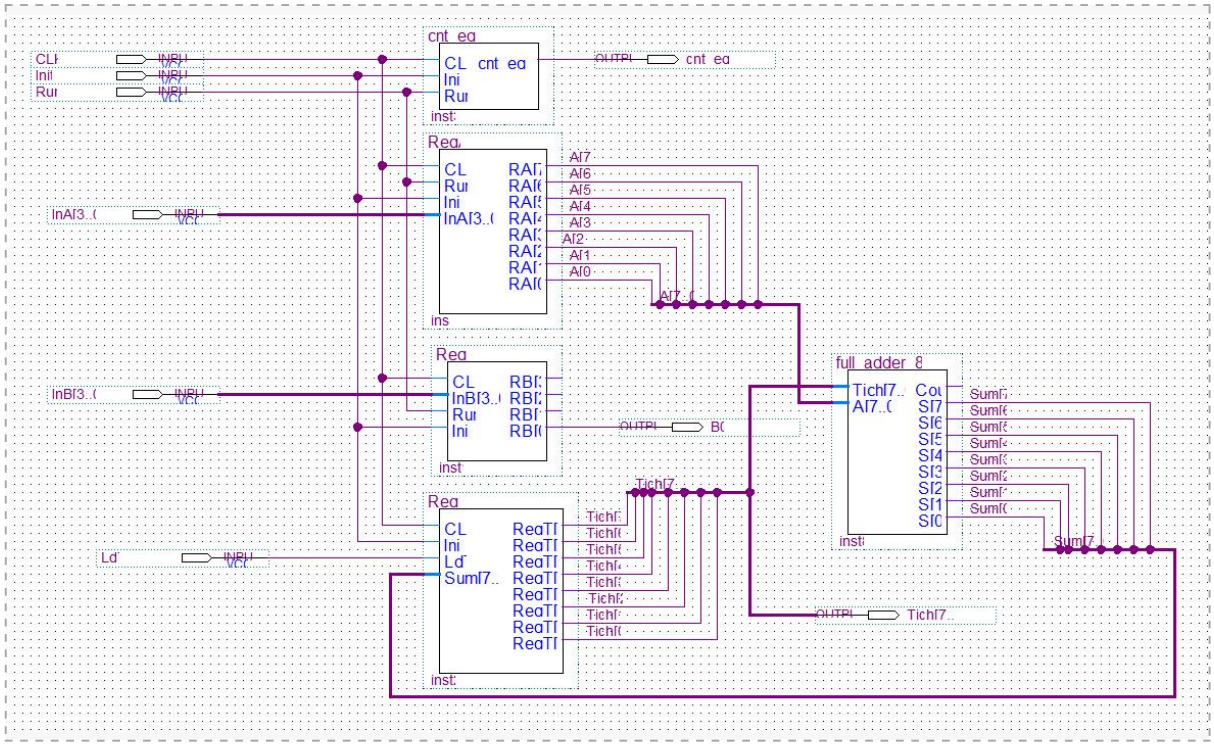


Hình 3.7: Bộ đếm vòng lặp (cnt_eq_3)

3.7. Datapath hoàn chỉnh

Bảng 3.6: Các chân vào/ra của datapath

Chân	Hướng	Độ rộng	Ý nghĩa
CLK	Vào	1	Xung clock chung
Init, Run, LdT	Vào	1	Tín hiệu điều khiển từ Control Unit
InA[3..0], InB[3..0]	Vào	4	Số bị nhân, số nhân
Tich[7..0]	Ra	8	Kết quả (ngõ ra RegT)
B0	Ra	1	Bit thấp nhất của B, gửi về Control Unit
cnt_eq_3	Ra	1	Báo cnt = 3, gửi về Control Unit



Hình 3.8: Datapath hoàn chỉnh

4. THIẾT KẾ CONTROL UNIT

Control Unit gồm 3 phần: **thanh ghi trạng thái** (2 D flip-flop, khối current), **logic trạng thái kế tiếp** (khối next_state) và **logic ngõ ra** (khối output_logic). Ngõ vào là Start, cnt_eq_3 và B0. Các ngõ ra Init, Run, Done chỉ phụ thuộc trạng thái (Moore); riêng LdT phụ thuộc thêm B0 (Mealy).

4.1. Bảng trạng thái kế tiếp

Bảng 4.1: Bảng trạng thái kế tiếp Q_1Q_0(next)

Trạng thái	Q_1Q_0	Start, cnt_eq_3 = 00	01	10	11
S0	00	00	00	01	01
S1	01	10	10	10	10
S2	10	10	11	10	11
S3	11	00	00	00	00

4.2. Phương trình trạng thái kế tiếp

Q_1(next) bằng 1 khi trạng thái kế tiếp là S2 hoặc S3, tức là khi đang ở S1 (luôn sang S2) hoặc S2 (ở lại S2 hoặc sang S3):

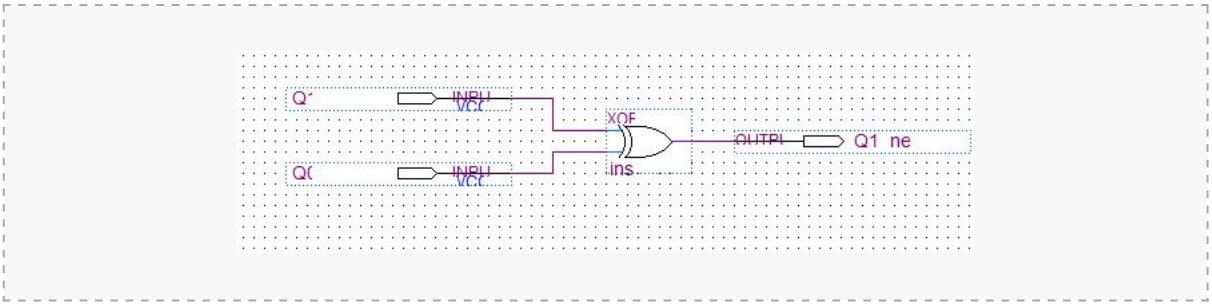
$$Q_1(next) = S1 + S2 = Q_1'Q_0 + Q_1Q_0' = Q_1 \oplus Q_0$$

Q_0(next) bằng 1 khi trạng thái kế tiếp là S1 hoặc S3, tức là khi đang ở S0 với Start = 1, hoặc ở S2 với cnt = 3:

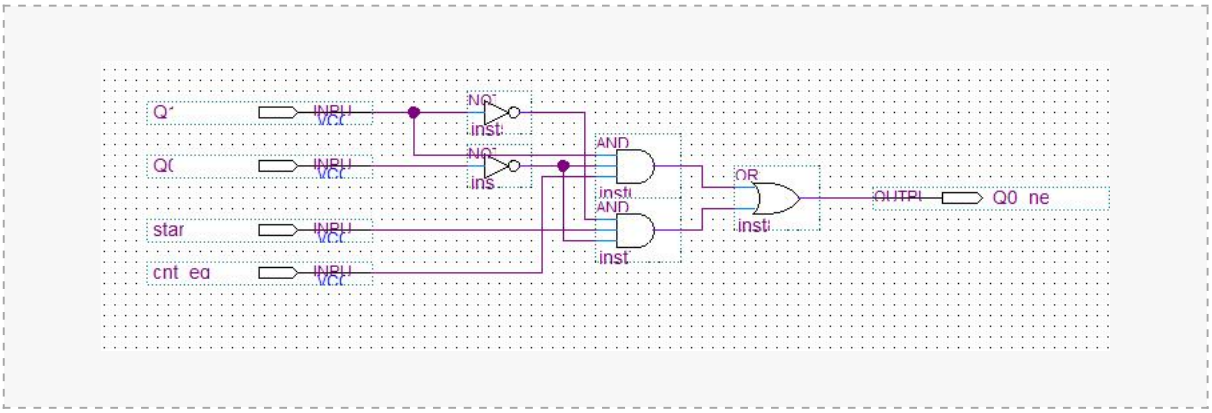
$$Q_0(next) = Q_1'Q_0'\cdot Start + Q_1Q_0'\cdot cnt_eq_3$$

Bảng 4.2: Kiểm tra phương trình trạng thái kế tiếp

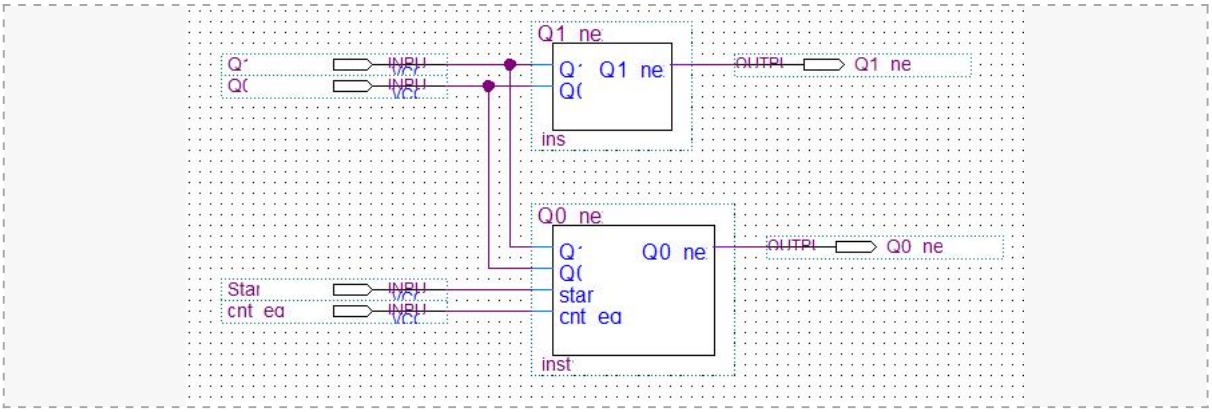
Hiện tại	Ngõ vào	Q_1(next)	Q_0(next)	Kế tiếp
S0 (00)	Start = 0	0	0	S0 ✓
S0 (00)	Start = 1	0	1	S1 ✓
S1 (01)	–	1	0	S2 ✓
S2 (10)	cnt_eq_3 = 0	1	0	S2 ✓
S2 (10)	cnt_eq_3 = 1	1	1	S3 ✓
S3 (11)	–	0	0	S0 ✓



Hình 4.1: Mạch Q1_next



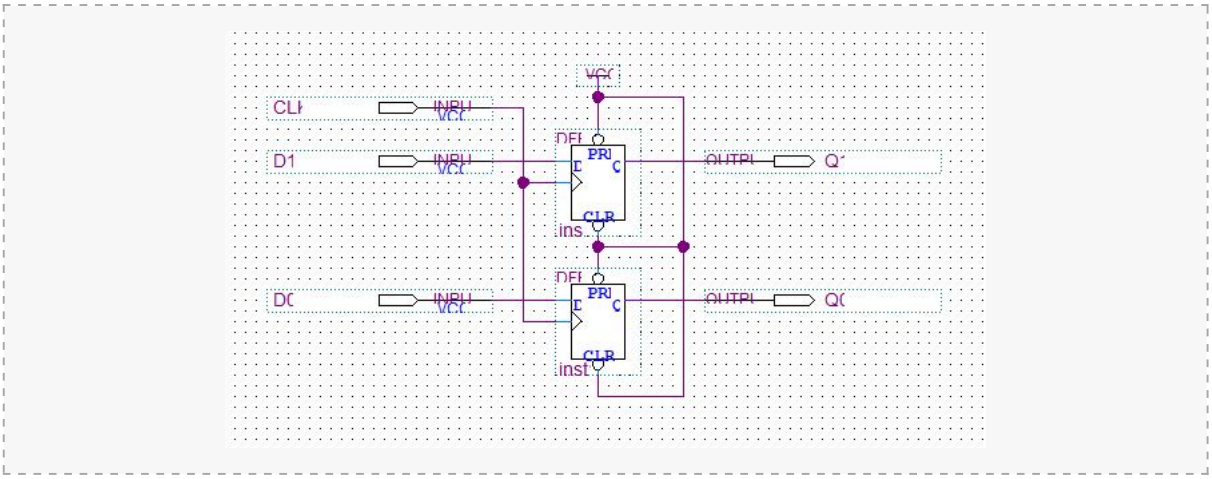
Hình 4.2: Mạch Q0_next



Hình 4.3: Khối logic trạng thái kế tiếp (next_state)

4.3. Thanh ghi trạng thái

Gồm 2 D flip-flop dùng chung CLK, PRN và CLRN nối VCC. Đầu vào D_1D_0 = Q_1Q_0(next), đầu ra Q_1Q_0 đưa đến output logic và hồi tiếp về next-state logic. Trong Quartus, flip-flop khởi động bằng 0 nên mạch bắt đầu ở S0 = 00.



Hình 4.4: Thanh ghi trạng thái (current)

4.4. Bảng ngõ ra

Bảng 4.3: Bảng ngõ ra theo trạng thái

Trạng thái	Q_1Q_0	Init	Run	LdT	Done
S0	00	0	0	0	0
S1	01	1	0	0	0
S2	10	0	1	**B0**	0
S3	11	0	0	0	1

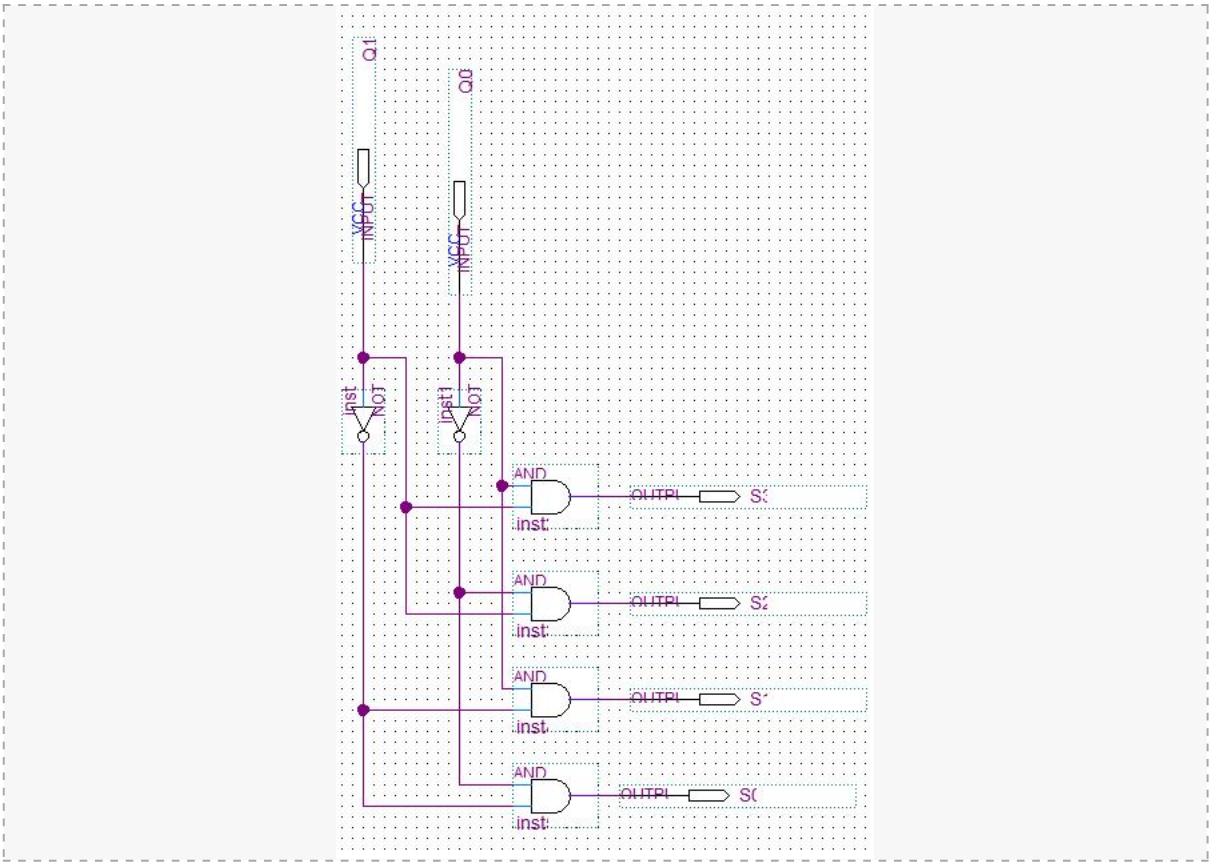
4.5. Phương trình ngõ ra

Trước hết giải mã trạng thái bằng khối **decoder_state** (2 cổng NOT, 4 cổng AND):

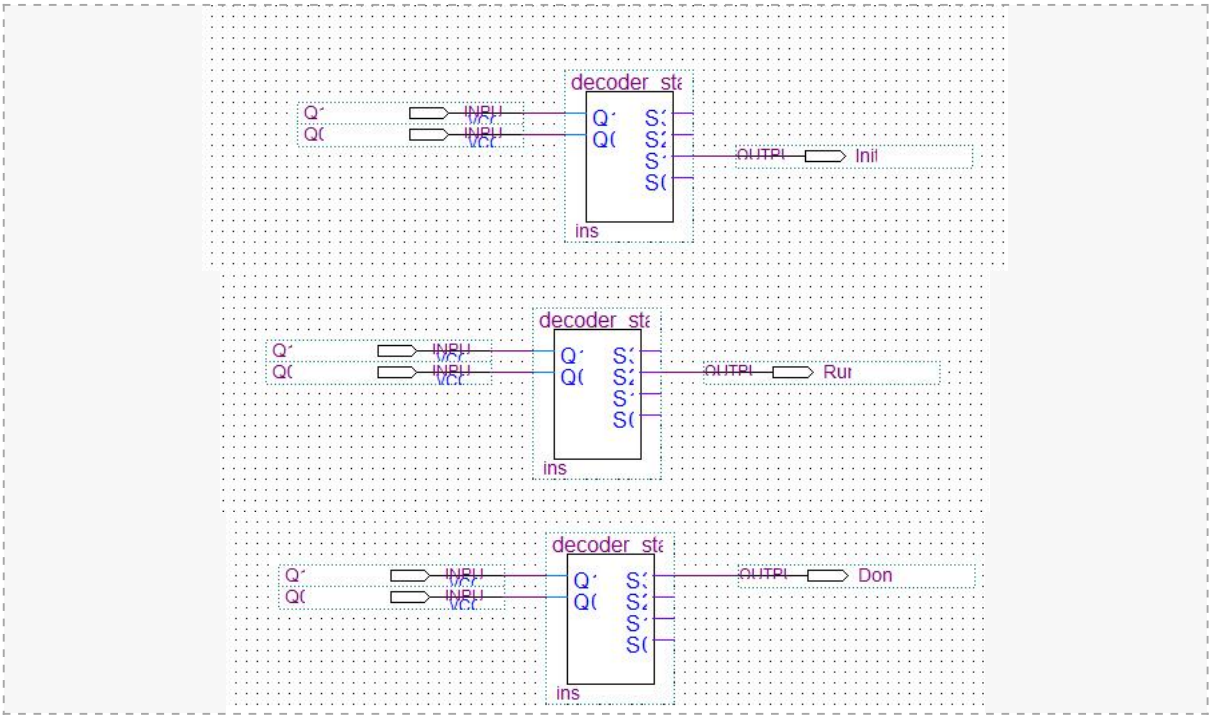
$$\mathbf{S0 = Q_1'Q_0'} \quad \mathbf{S1 = Q_1'Q_0} \quad \mathbf{S2 = Q_1Q_0'} \quad \mathbf{S3 = Q_1Q_0}$$

Bảng 4.4: Phương trình logic ngõ ra

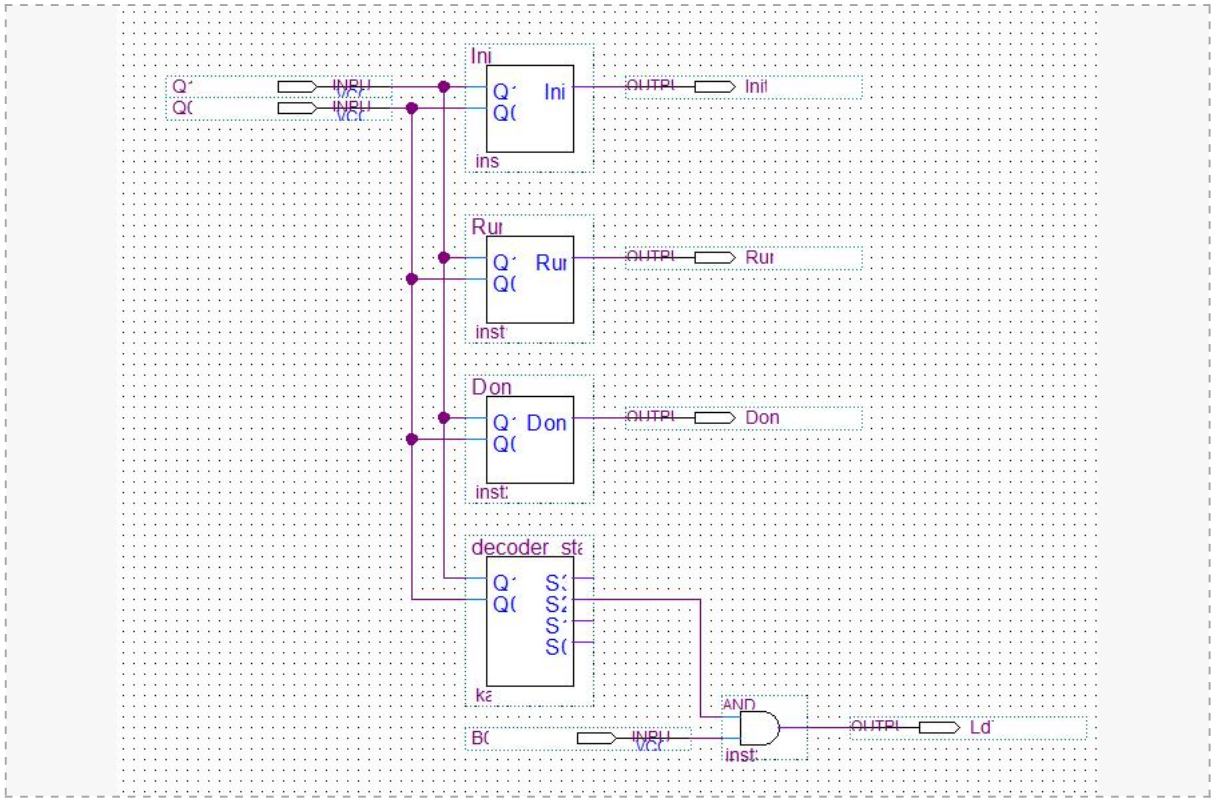
Tín hiệu	Phương trình	Loại	Khối
Init	$S1 = Q_1'Q_0$	Moore	Init
Run	$S2 = Q_1Q_0'$	Moore	Run
LdT	$S2 \cdot B0 = Q_1Q_0' \cdot B0$	Mealy	AND trong output_logic
Done	$S3 = Q_1Q_0$	Moore	Done



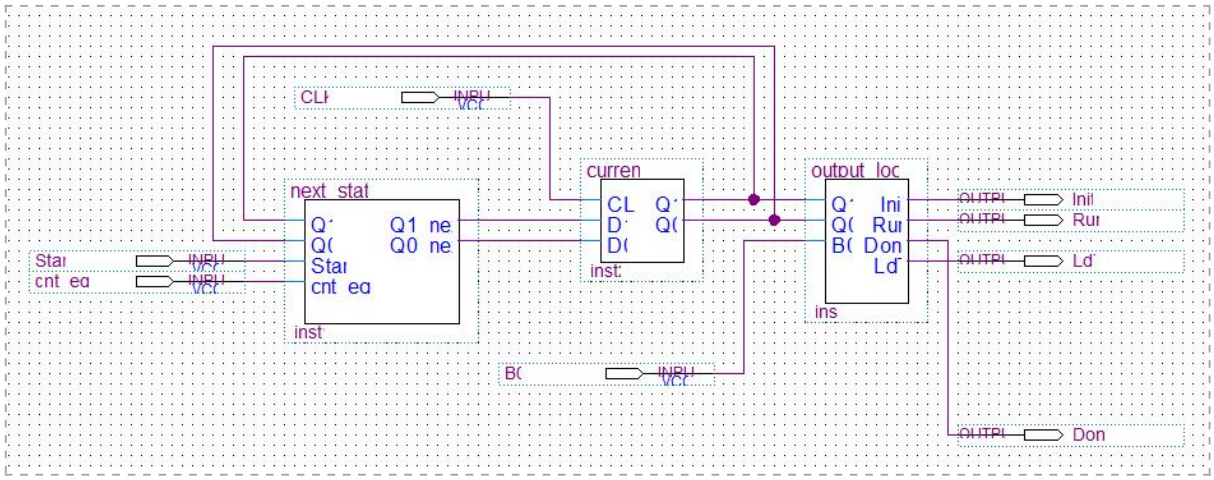
Hình 4.5: Bộ giải mã trạng thái (decoder_state)



Hình 4.6: Các khối Init, Run, Done



Hình 4.7: Mạch logic ngõ ra (output_logic)



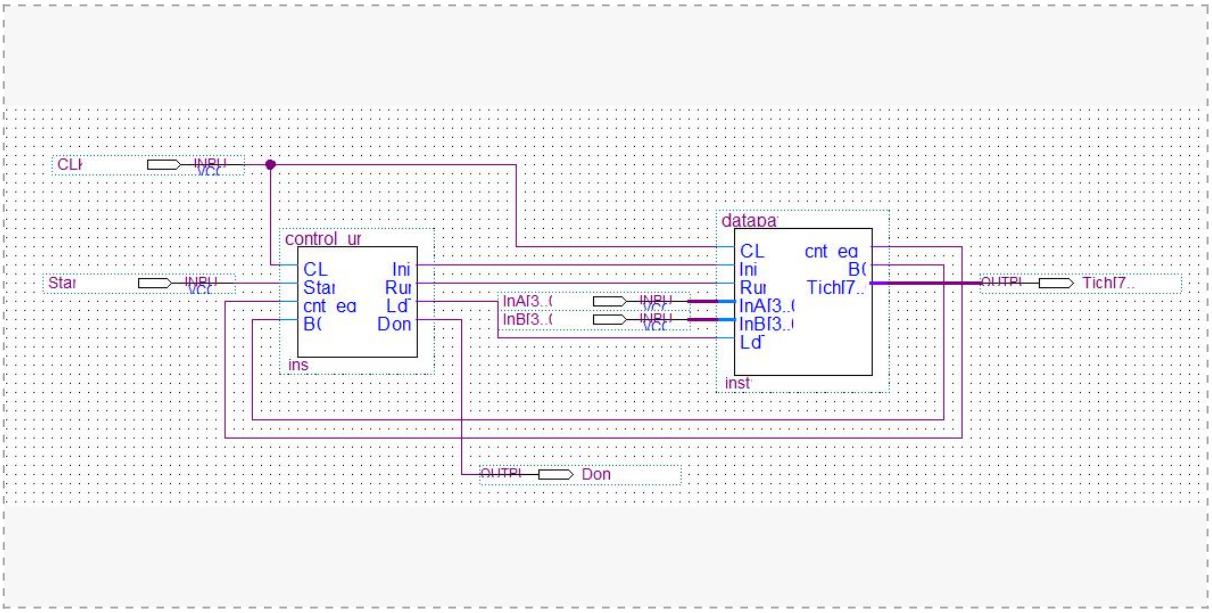
Hình 4.8: Control Unit hoàn chỉnh (control_unit)

5. MẠCH TỔNG HOÀN CHỈNH

Mạch tổng **nhan_2_so** ghép Control Unit với Datapath. Init, Run, LdT đi từ Control Unit sang Datapath; B0 và cnt_eq_3 đi ngược từ Datapath về Control Unit; CLK dùng chung cho cả hai khối.

Bảng 5.1: Các chân vào/ra của mạch tổng

Chân	Hướng	Độ rộng	Ý nghĩa
CLK	Vào	1	Xung clock hệ thống
Start	Vào	1	Bắt đầu phép nhân (đọc ở S0)
InA[3..0]	Vào	4	Số bị nhân, nạp vào RegA ở S1
InB[3..0]	Vào	4	Số nhân, nạp vào RegB ở S1
Tich[7..0]	Ra	8	Tích (ngõ ra RegT)
Done	Ra	1	Báo kết quả hợp lệ (= 1 ở S3)



Hình 5.1: Mạch tổng nhân hai số 4 bit (Control Unit + Datapath)

> **Về Done:** Tich là ngõ ra trực tiếp của thanh ghi Tích nên trong quá trình nhân sẽ hiển thị các **tích riêng phần** (ví dụ 0 → 26 → 130 với 13 × 10). Giá trị đọc tại chu kỳ Done = 1 mới là kết quả cuối cùng.

6. MÔ PHỎNG VÀ KIỂM CHỨNG

6.1. Thiết lập mô phỏng

- CLK chu kỳ **T = 20 ns** (cạnh lên tại 10, 30, 50, … ns). End Time: 1 µs.
- Lần nhân 1: Start = 1 trong khoảng 20–40 ns (nhận tại cạnh lên 30 ns); InA, InB giữ ổn định trong 20–60 ns.
- Lần nhân 2: Start = 1 trong khoảng 160–180 ns (nhận tại cạnh 170 ns); InA, InB giữ trong 160–200 ns.
- Start, InA, InB chỉ đổi giá trị tại **cạnh xuống** của CLK, nên tại cạnh lên tín hiệu đã ổn định (không vi phạm setup khi mô phỏng Timing).

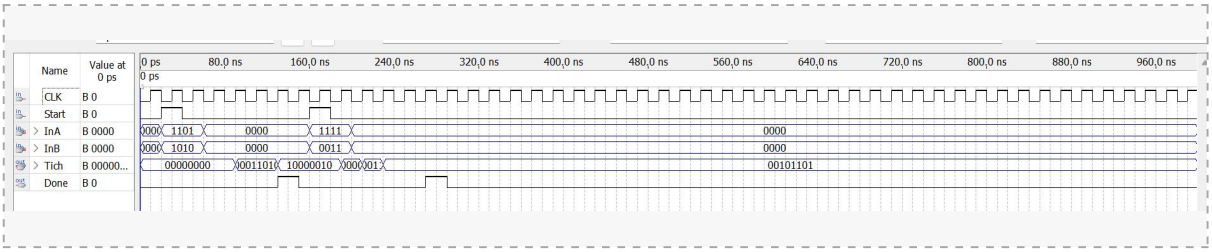
Lưu ý: InA, InB phải được giữ ổn định qua cạnh clock kết thúc S1 (cạnh 50 ns ở lần nhân 1), vì RegA và RegB chỉ nạp dữ liệu tại cạnh này. Chạy hai lần nhân liên tiếp còn giúp kiểm tra Tích và cnt được xóa đúng ở S1 (không cộng dồn kết quả cũ).

Bảng 6.1: Các trường hợp kiểm tra

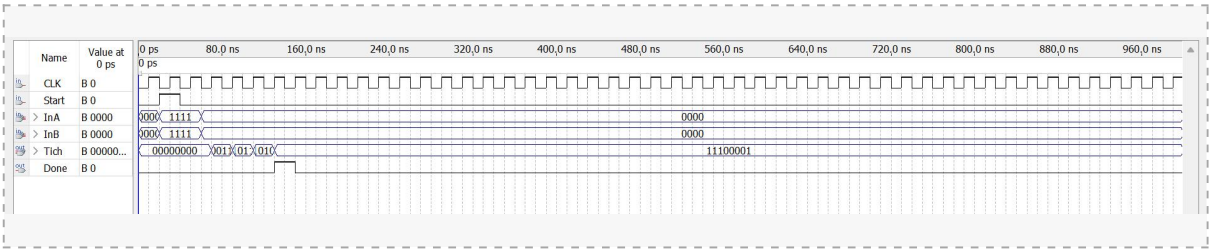
InA	InB	Tích mong đợi	Tích mô phỏng	Kết quả
1101 (13)	1010 (10)	1000 0010 (130)	1000 0010	Đúng
1111 (15)	0011 (3)	0010 1101 (45)	0010 1101	Đúng
1111 (15)	1111 (15)	1110 0001 (225)	1110 0001	Đúng

6.2. Mô phỏng Functional

Mô phỏng chức năng bỏ qua độ trễ, dùng để kiểm tra thuật toán và luồng trạng thái.



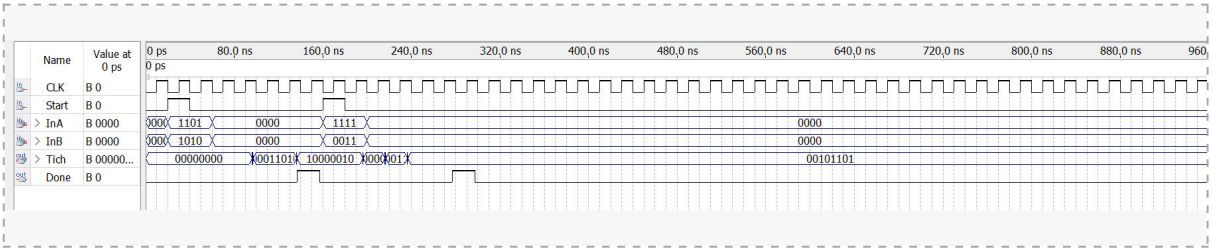
Hình 6.1: Waveform Functional – 13 × 10 và 15 × 3



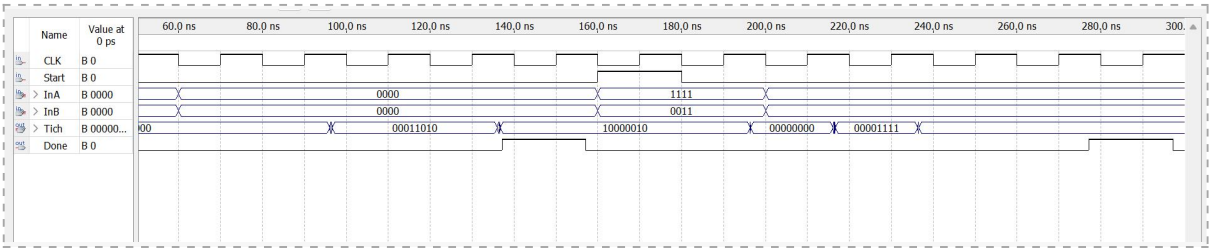
Hình 6.2: Waveform Functional – 15 × 15

6.3. Mô phỏng Timing

Mô phỏng timing dùng độ trễ thực của cổng logic và dây nối sau bước Fitter, nên tín hiệu ra đổi sau cạnh clock một khoảng và có thể xuất hiện glitch.



Hình 6.3: Waveform Timing – 13 × 10 và 15 × 3



Hình 6.4: Phóng to độ trễ clock-to-output

6.3.1. Độ trễ clock-to-output

Đo khoảng thời gian từ cạnh lên CLK đến khi Tich bắt đầu đổi giá trị: cạnh lên tại ≈ 130 ns, Tich đổi tại ≈ 138 ns → độ trễ $\mathbf{t_{co} \approx 8\ ns}$. Độ trễ này cần nhỏ hơn chu kỳ 20 ns để mạch hoạt động đúng ở 50 MHz.

6.3.2. Phân tích glitch

Khi nhiều bit của Tich đổi cùng lúc, mỗi bit có độ trễ đường truyền khác nhau nên có thể xuất hiện giá trị trung gian rất ngắn:

Bảng 6.2: Các lần Tich đổi giá trị (13 × 10)

Chuyển	Nhị phân	Số bit đổi	Glitch quan sát được
0 → 26	0000 0000 → 0001 1010	3 (bit 4, 3, 1)	00001000, 00001010
26 → 130	0001 1010 → 1000 0010	3 (bit 7, 4, 3)	00010010, 10010010

Glitch không ảnh hưởng kết quả vì hệ thống đồng bộ: các thanh ghi chỉ lấy mẫu tại cạnh clock (khi tín hiệu đã ổn định) và kết quả được đọc khi Done = 1.

6.3.3. So sánh Functional và Timing

Bảng 6.3: So sánh mô phỏng Functional và Timing

Tiêu chí	Functional	Timing
Độ trễ	Bằng 0, đổi đúng tại cạnh clock	Có (≈ 8 ns clock-to-output)

Tiêu chí	Functional	Timing
Glitch	Không có	Có thể có khi nhiều bit đổi cùng lúc
Kết quả cuối	13 × 10 = 130; 15 × 3 = 45; 15 × 15 = 225	13 × 10 = 130; 15 × 3 = 45
Số chu kỳ	6	6
Mục đích	Kiểm tra thuật toán	Kiểm tra khả năng chạy trên chip thật

6.4. Số chu kỳ hoàn thành

Quy ước: tính từ chu kỳ đầu tiên sau khi nhận Start (vào S1) đến hết S3 (Done = 1). Trạng thái chờ S0 không tính.

Bảng 6.4: Diễn biến từng chu kỳ với 1101 × 1010

Chu kỳ	Thời gian	Trạng thái	cnt	B	B0	Tich (sau cạnh cuối chu kỳ)
1	30 – 50 ns	S1	–	–	–	0000 0000 (A, B được nạp)
2	50 – 70 ns	S2	0	1010	0	0000 0000
3	70 – 90 ns	S2	1	0101	1	0001 1010 (26)
4	90 – 110 ns	S2	2	0010	0	0001 1010 (26)
5	110 – 130 ns	S2	3 → thoát	0001	1	1000 0010 (130)
6	130 – 150 ns	S3	–	–	–	Done = 1, Tich = 130

Bảng 6.5: Đếm số chu kỳ

Giai đoạn	Trạng thái	Số chu kỳ	Thời gian (T = 20 ns)
Khởi tạo	S1	1	30 – 50 ns
Vòng lặp	4 × S2	4	50 – 130 ns
Xuất kết quả	S3	1	130 – 150 ns
Tổng		**6**	**120 ns = 6 × 20 ns**

Số chu kỳ = 1 (S1) + 4 (S2) + 1 (S3) = 6 chu kỳ

Nhận xét: số chu kỳ **cố định bằng 6**, không phụ thuộc giá trị InA, InB, vì vòng lặp luôn chạy đúng 4 lần (theo cnt) và mỗi vòng chỉ tốn 1 chu kỳ nhờ gộp phép cộng (Mealy) với phép dịch vào cùng trạng thái S2. Nếu thiết kế theo kiểu Moore (tách kiểm tra, cộng, dịch thành các trạng thái riêng) thì mỗi vòng tốn 2–3 chu kỳ và tổng số chu kỳ là 10–14, phụ thuộc số bit 1 của B.

Bảng 6.6: So sánh số chu kỳ hai cách thiết kế ASM

Cách thiết kế	Số trạng thái	Chu kỳ / vòng	Tổng số chu kỳ
Moore (TEST, ADD, SHIFT tách riêng)	6	2 hoặc 3	10 – 14 (phụ thuộc B)
Mealy (gộp vào S2) – thiết kế này	4	1	6 (cố định)

7. KẾT LUẬN

- Đã thiết kế hoàn chỉnh mạch tuần tự nhân hai số nhị phân 4 bit theo mô hình FSMD trên Quartus, vẽ hoàn toàn bằng cổng logic và D flip-flop: lưu đồ ASM 4 trạng thái, datapath (RegA, RegB, RegT, bộ cộng 8 bit, bộ đếm cnt) và Control Unit (next_state, current, output_logic).
- Mô phỏng Functional và Timing đều cho kết quả đúng: 13 × 10 = 130, 15 × 3 = 45 (Functional thêm 15 × 15 = 225, trường hợp lớn nhất); hai lần nhân liên tiếp cho thấy Tích và cnt được xóa đúng ở S1.
- Số chu kỳ hoàn thành cố định **6 chu kỳ** (120 ns với T = 20 ns), không phụ thuộc dữ liệu vào.
- Các thanh ghi dùng **xóa đồng bộ** qua chân D nên không bị ảnh hưởng bởi glitch của mạch giải mã trạng thái khi mô phỏng Timing.
- **Ưu điểm:** nhanh nhờ gộp vòng lặp vào một trạng thái (Mealy), Control Unit đơn giản (2 flip-flop, 4 tín hiệu điều khiển). **Nhược điểm:** tốn phần cứng hơn do A và bộ cộng rộng 8 bit. Có thể cải tiến bằng cách dịch phải khối {C, Acc, Q} để chỉ cần bộ cộng 4 bit, hoặc dừng sớm khi B = 0 để giảm số chu kỳ với số nhân nhỏ.